

2026-04-22--2026-05-06

1. 论文

1.1 Document-level relation extraction with hierarchical dependency tree and bridge path

1.1.1 问题

主要针对于文档级别RE,先前的方法存在：

1. 建模粒度粗：把整篇文章直接丢进图神经网络，粒度粗，句子内部细节信息丢失，且BERT有512token的长度限制。
2. context被忽视：previous方法过度关注实体本身的语义，忽略了上下文。
3. 上下文一视同仁：实际上和头尾实体共同出现的信息才是关系的关键，而不是将上下文都平等对待。

1.1.2 方法：HDT-BP

即Hierarchical (分层的) Dependency Tree + Bridge Path。

1.1.2.1 句子级别选代表

首先以句子为单位进行token嵌入，逐句生成特征。

$$n_{mi}^1 = \{max(\{v_{jk}\}_{j=s}^e)\}_{k=1}^d$$

1. n_{mi}^1 代表第1个句子里第*i*个实体提及（mention）的节点特征
2. v_{jk} 是从起始位置*s*到结束位置*e*的词向量序列
3. 取每个维度的最大值，变成一个代表向量

[CLS] Emerson Samuels was born here.

对于上述句子，实体提及（mention）就是“Emerson Samuels”这个序列，即作者使用了max-pooling来将实体提及进行融合。

然后就是分别取mention中这两个词，也就是Emerson和Samuels，这两个单词的向量

例如Emerson=[0.9, 0.1, 0.2]

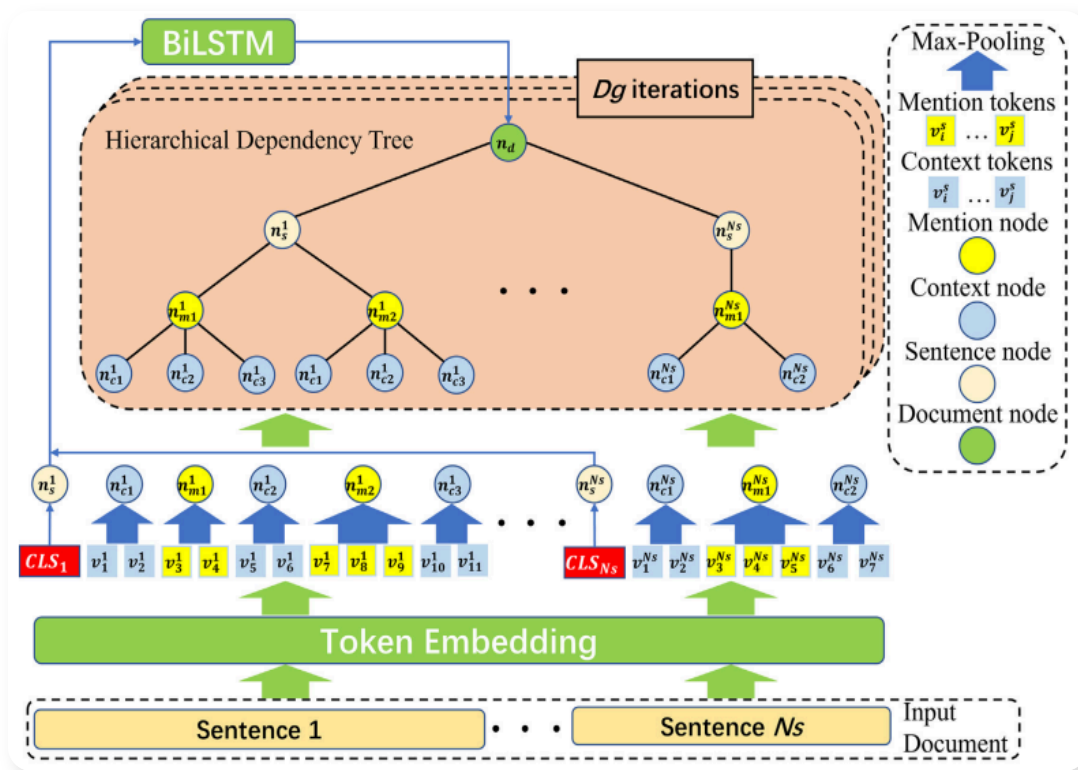
Samuels=[0.7, 0.2, 0.1]

接着针对每个维度选最大值，就得到了所谓的这个实体的**特征节点 (mention node)**，也就是[0.9, 0.2, 0.2]，当然CNN的max-pooling比这个复杂，但本质是取最大值，目的是降维、对齐，并保留局部最显著的特征

接着利用BiLSTM将所有句子的[CLS]穿起来，形成一个代表整篇文章的全局文档节点 (Document Node)。

1.1.2.2 构建HDT

构建了4个层级的树状图：文档层 -> 句子层 -> mention层 -> 上下文层。



MentionNode、ContextNode的计算就是第一部说的max-pooling方法。然后用BERT形成SentenceNode；用BiLSTM形成DocumentNode。

再使用图卷积网络来迭代训练，不同类型，层级的节点使用不同的矩阵。对于最后一项也就是文档节点，实际上如果你不是句子节点，那这一项的结果就是0。也就是一层层听人讲话。

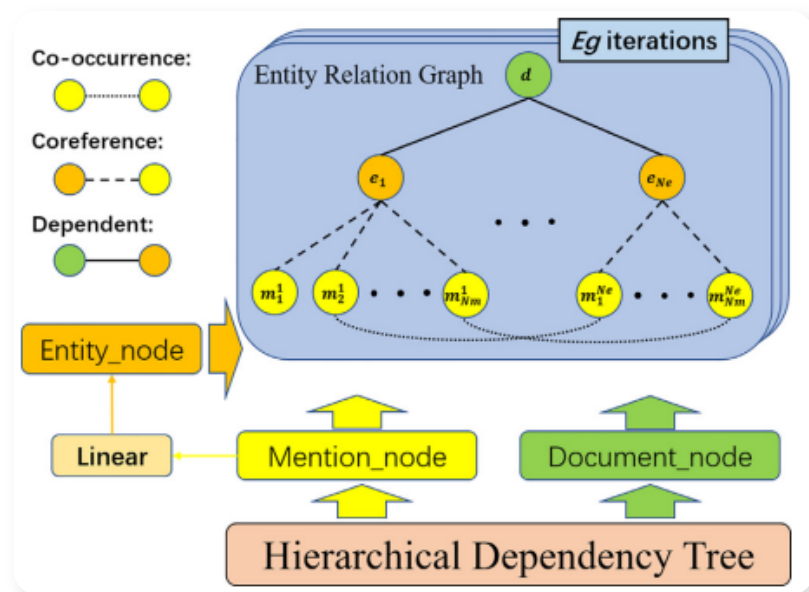
$$n_i^{l+1} = ReLU(\underbrace{\sum_{j \in N_{cm}(i)} (W_{cm} n_j^l)}_{\text{第一块: 上下文 (下属)}} + \underbrace{\sum_{j \in N_{ms}(i)} (W_{ms} n_j^l)}_{\text{第二块: 句子 (上司)}} + \underbrace{\sum_{j \in N_{sd}(i)} (W_{sd} n_j^l)}_{\text{第三块: 文档节点}} + b_h)$$

1.1.2.3 实体相关图ECG

经过HDT之后，MentionNode和DocumentNode已经充分吸收了context和sentence的特征。但他有一个问题，在HDT的眼里，对于MentionNode，他不懂**共同指向**。例如前一句有乔布斯，后一句有他，很明显这是共同指向关系，但在上述的分层树结构中，HDT是无法知道这件事的，因为他们只是不同树枝上的叶子。

所以引入ECG (entity feature modeling)，将所有共同指向的词语合并成一个真正的实体。操作也很简单，就是直接向量相加，也就是下图中橙色的节点，也就是真正的实体概念。

$$e_i^0 = W_m \sum_{j=1}^{N_m} m_j^i + b_m$$



1. Co-occurrence：黄色节点之间的虚线，也就是句子中一个实体和另一个实体之间存在联系，给他们连线，因为他们现在被放在不同的群里了。
2. Coreference：共同指代，都是指代一个东西。
3. Dependent：文档和真正实体的依赖关系。

再通过图卷积网络迭代。相当于通过共同出现和共同指代，拉了一张全新的关系网，将本来可能相隔很远的实体拉了起来。

1.1.2.4 BridgePath

如果头实体 e_h 和尾实体 e_t 没有在一个句子同框过该怎么办？这个时候需要找一个Bridge Entity e_q 。如果实体 e_q 既和头实体 e_h 在同一个句子里出现过，又和尾实体 e_t 出现过，那么它就是桥接实体。

接着把头实体、桥实体、尾实体依次喂给BiLSTM。得到的隐藏状态就是这条路径的特征代表。

最后把头实体特征、尾实体特征、桥接路径特征和DocumentNode特征丢给softmax。

1.1.3 结果

在目前最大的篇章级关系抽取数据集DocRED上达到SOTA。

1.1.4 参考

其通过max-pooling方法得到MentionNode的思路后续可以借鉴。

1.2 Leveraging shortest dependency paths in low-resource biomedical relation extraction

1.2.1 问题

1. 低资源场景
2. 半监督等方法虽能利用未标注的数据，但也常常引入噪声。

1.2.2 方法

1.2.2.1 更强的SDP表示

$$SDP_{rep} = h_s \oplus \left(\frac{1}{|SDP|} \sum_{w_i \in SDP} h_i \right) \oplus h_o$$

1. h_s 和 h_o 分别是头实体和尾实体。
2. $\frac{1}{|SDP|} \sum_{w_i \in SDP} h_i$ ，即把SDP上剩余的词加权求平均。
3. 最后再进行向量拼接，得到一个包含最核心的头尾实体以及核心关系的表征向量。

1.2.2.2 KNN近邻传播与软标签

1.2.2.2.1 KNN

对于任意一个无标签样本 x_u ，首先通过编码器获取它的SDP表征向量 $v_u = SDP_{rep}(x_u)$ ；然后在已标注数据集 $\mathcal{D}_L$ 中，计算与所有已标注样本表征 v_l 之间的余弦相似度。

$$d(v_u, v_l) = \frac{v_u \cdot v_l}{\|v_u\| \|v_l\|}$$

然后在提取出距离最近的Top-k个邻居样本，论文里设置为5，即KNN。

1.2.2.2.2 按类别相似度

找到这k个最近的邻居之后，将它们按照真实的类别进行分组，并把余弦相似度累加起来。

$$S_c = \sum_{j \in N_k(x_u) \cap C_j=c} d(v_u, v_j)$$

也就是分组之后，再去看看和每个分组里的那些向量的余弦相似度的累加和，就得到 x_u 和这个分组的相似度。

1.2.2.2.3 softmax

$$y_{u,c} = \frac{\exp(S_c)}{\sum_{c'=1}^K \exp(S_{c'})}$$

再softmax归一化。

1.2.2.2.4 损失函数

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^K y_{ic} \log \hat{y}_{ic}$$

即交叉熵损失函数。

1.2.2.3 将SDP骨架作为prompt

也就是利用上述的算法，在prompt中搜索出与待预测句子的SDP骨架最相似的案例，作为参考丢给大模型。

1.2.3 结果

当只有500个标注数据时，对比传统核方法，在PPI数据集的F1分数从54.7%提升到了79.4%。在DDI数据集上，使用这种方法加上100个标注数据，效果超越了传统方法使用500个标注数据的表现。

这篇论文的检索思路可以作为参考，在我后续的检索中可以参考参考。

2. 实验

加入syntax-view与semantics-views的结果较好

2.1 ChemProt

model	eval loss	mapped exact
B2-coarse	0.058921	0.866946

model	eval loss	mapped exact
B3-syntax-only	0.122404	0.899948
B4-semantics-only	0.084065	0.901083
B5-dual-view	0.083922	0.896106

ChemProtSent当前最优的是B4-semantics-only。

2.2 CDR

模型	eval loss	mapped exact
Stage 2 B2-coarse seed42	0.107883	0.713408
B3 syntax-only	0.140148	0.764695
B4 semantics-only	0.221137	0.769305
B5 dual-view concat	0.251782	0.775451

CDRIntra当前最优的是B5-dual-view。

2.3 下一步

参考multi-view论文，CDRIntra上B5最好，说明syntax-view和semantics-view确实有效，ChemProtSent上B4最好，将继续加入信息筛选模块。